Shiv Nadar University Chennai

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

Experiment 1
Implementation of a Single Layer Perceptron for Binary Classification

1. Objective

The objective of this experiment is to understand the concept of an artificial neuron and implement a Single Layer Perceptron from scratch for binary classification. Students will learn the perceptron learning algorithm, understand the importance of activation functions, visualize the learning process, and evaluate the classifier using a real-world dataset.

2. Background Theory

An Artificial Neuron is the fundamental building block of a neural network. It receives one or more input features, multiplies them by corresponding weights, adds a bias term, and applies an activation function to determine the output.

The Single Layer Perceptron is the earliest supervised learning algorithm capable of solving linearly separable binary classification problems.

Mathematical Formulation

Weighted Sum

$$z = \mathbf{w}^T\mathbf{x} + b$$

where

- $\mathbf{x}$: Input vector
- $\mathbf{w}$: Weight vector
- b : Bias
- z : Linear combination

Prediction

$$\hat{y} = f(z)$$

Weight Update Rule

$$\mathbf{w}_{new} = \mathbf{w}_{old} + \eta(y - \hat{y})\mathbf{x}$$

Bias Update

$$b_{new} = b_{old} + \eta(y - \hat{y})$$

where η denotes the learning rate.

3. Activation Functions

An activation function determines the output of a neuron based on the weighted sum of its inputs. It introduces non-linearity, enabling neural networks to learn complex patterns. Although the perceptron uses only the Step Activation Function, modern deep learning models employ differentiable activation functions for efficient optimization through backpropagation.

Step Activation Function

$$f(z) = \begin{cases} 1, & z \geq 0 \\ 0, & z < 0 \end{cases}$$

The Step Activation Function produces a binary output and is therefore suitable only for linearly separable binary classification problems.

Activation	**Output Range**	**Typical Applications**
Step	$\{0, 1\}$	Classical Perceptron
Sigmoid	$(0, 1)$	Binary Classification Output Layer
Tanh	$(-1, 1)$	Hidden Layers (Traditional Networks)
ReLU	$[0, \infty)$	Hidden Layers in CNNs and MLPs
Leaky ReLU	$(-\infty, \infty)$	Avoiding Dying ReLU
Softmax	$(0, 1)$	Multi-class Classification Output Layer

Note: This experiment uses only the Step Activation Function. The remaining activation functions will be studied in subsequent experiments involving Multilayer Perceptrons and Deep Neural Networks.

4. Dataset Description

Dataset: Banknote Authentication Dataset

Source: UCI Machine Learning Repository

Download Link:

`https://archive.ics.uci.edu/dataset/267/banknote+authentication`

Instances	1372
Features	4 Numerical Features
Classes	2
Missing Values	None
Task	Binary Classification

Feature Description

- Variance
- Skewness
- Curtosis
- Entropy

Target Class

- 0 – Authentic Banknote
- 1 – Forged Banknote

5. Experimental Procedure

Task 1: Dataset Exploration

- Load the dataset.
- Display the first five samples.
- Determine dataset dimensions.
- Identify missing values.
- Display descriptive statistics.

Task 2: Exploratory Data Analysis
Generate

- Histograms
- Correlation Heatmap
- Scatter Plot
- Boxplots

Task 3: Data Preprocessing

- Normalize all numerical features.

- Split the dataset into Training (80%) and Testing (20%).

Task 4: Perceptron Implementation
Implement from scratch

- Weight Initialization
- Bias Initialization
- Step Activation Function
- Forward Propagation
- Perceptron Learning Rule

Task 5: Model Training
Display

- Epoch Number
- Misclassified Samples
- Updated Weights
- Updated Bias

Task 6: Model Evaluation
Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

6. Source Code

The complete implementation (data loading, EDA, preprocessing, the from-scratch Perceptron class, training loop, evaluation, and learning-rate comparison) is available at the following GitHub repository:

`https://github.com/PolarFox08/Deep-Learning-Lab/blob/main/Deep%20Learning%20Lab%201/DL_Lab_1_Perceptron.ipynb`

7. Experimental Results

The perceptron was trained for 50 epochs with a learning rate of $\eta = 0.01$ on the standardized/normalized features, using an 80:20 train–test split (1097 training samples, 275 testing samples).

During training, the number of misclassified samples per epoch dropped sharply from 166 (epoch 1) to under 20 within the first 10 epochs, and then oscillated in a low range (roughly 14–26 misclassified samples) for the remaining epochs without reaching exactly zero. This indicates the model did not achieve perfect convergence, which is expected since the Banknote Authentication dataset is only *approximately* linearly separable (as seen in the EDA boxplots and scatter plots, where curtosis and entropy show considerable overlap between classes).

The final trained model (epoch 50) had:

- **Weights:** $[-0.181,\ -0.180,\ -0.208,\ 0.006]$ (variance, skewness, curtosis, entropy)
- **Bias:** 0.240

On the held-out test set, the model achieved:

Metric	**Value**
Accuracy	0.9891
Precision	1.0000
Recall	0.9764
F1-score	0.9880

A precision of 1.0000 indicates zero false positives – every banknote predicted as authentic was genuinely authentic. The small gap between precision and recall (0.9764) indicates a few forged notes were misclassified as authentic (false negatives), which is what pulled the F1-score slightly below 1.

Comparison with Scikit-learn's Perceptron

Implementation	**Test Accuracy**
From-scratch Perceptron	0.98909
Scikit-learn Perceptron	0.97818

The from-scratch implementation slightly outperformed scikit-learn's built-in `Perceptron` on this split. This is not evidence that the manual implementation is fundamentally superior – the two are not doing identical computations. Scikit-learn's `Perceptron` shuffles the training data at the start of every epoch by default and applies an early-stopping criterion (`tol`, `n_iter_no_change`), whereas the from-scratch version processes samples in a fixed order for a fixed 50 epochs. These implementation-level differences lead to slightly different final decision boundaries, so small accuracy differences of a point or two in either direction are expected on a small (275-sample) test set, not a sign of error.

8. Generated Plots with Interpretation

8.1 Feature Histograms

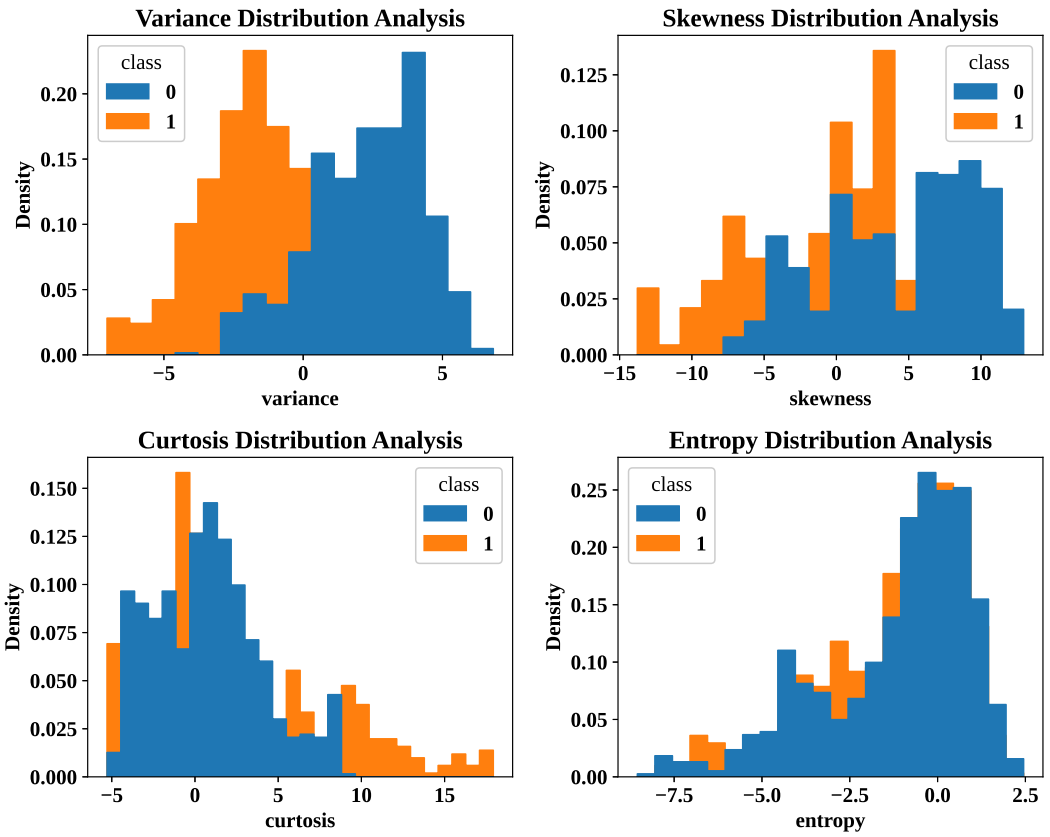

Figure 1: Distribution of each feature (variance, skewness, curtosis, entropy) split by class.

Interpretation: It is clear from the histogram plots that variance is the only symmetric and uniform distribution, with noticeable separation between the 2 classes. Curtosis and Entropy are extremely skewed and hence have extreme outlier values. The skewness distribution isn't as skewed as curtosis or entropy but still is slightly negatively skewed.

8.2 Correlation Heatmap

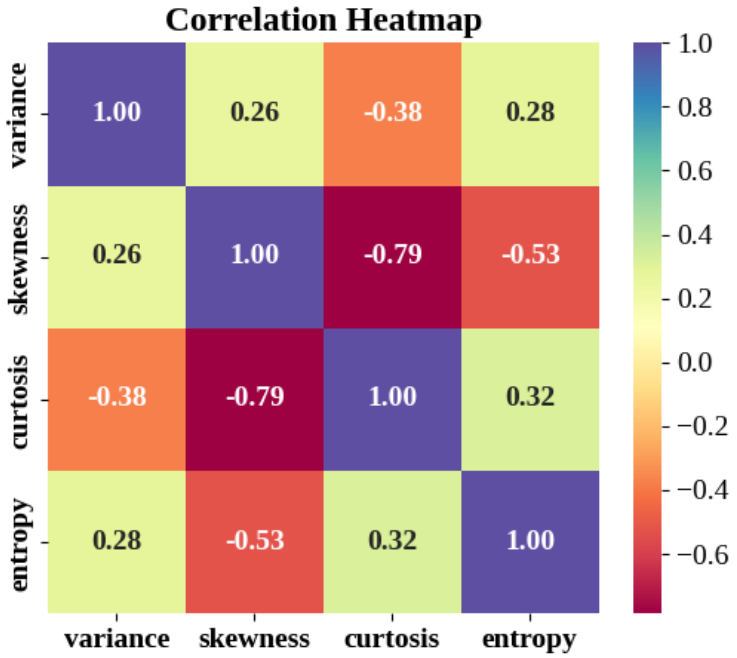

Figure 2: Pairwise correlation between the four input features.

Interpretation: Skewness and Curtosis are highly negatively correlated meaning curtosis decreases as skewness increases. At the same time there is a significant amount of negative correlation between skewness and entropy too.

8.3 Scatter Plot

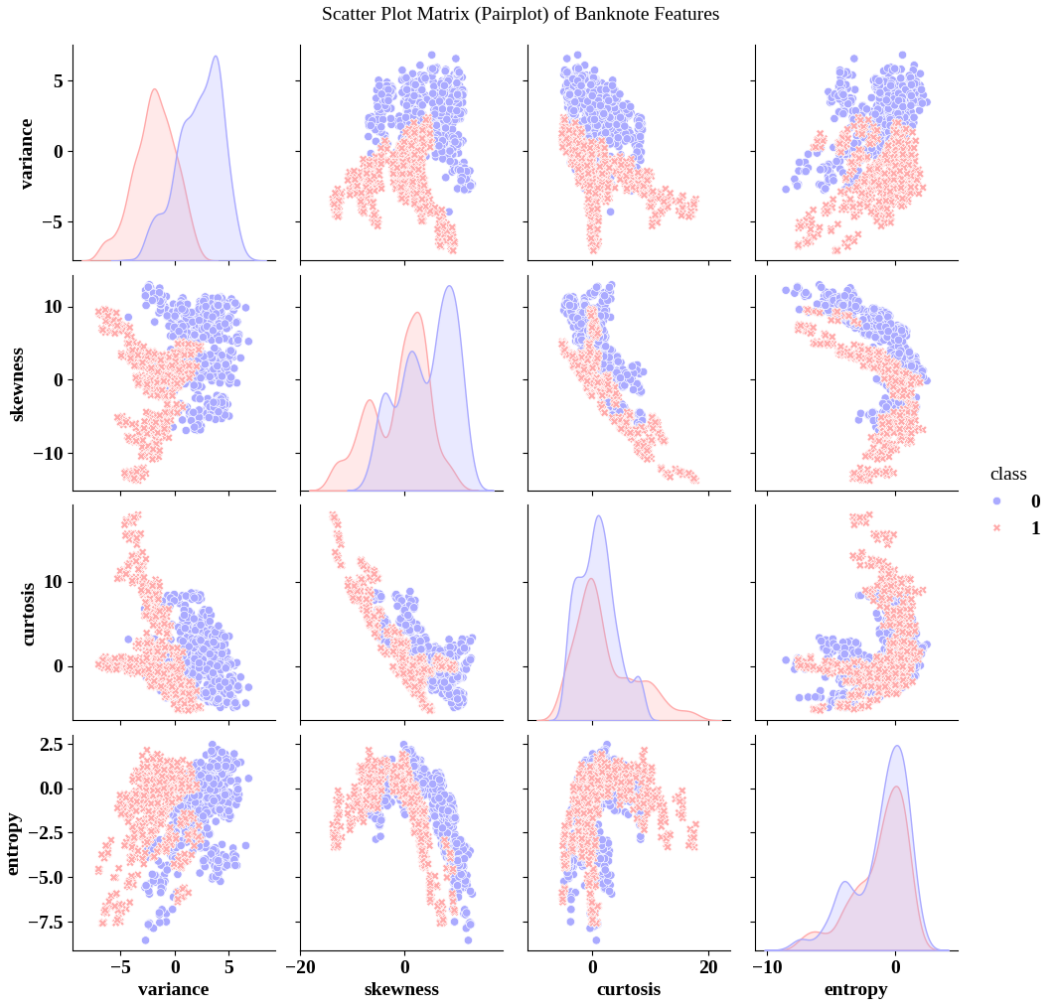

Figure 3: Pairwise scatter plot / scatter matrix of features colored by class.

Interpretation: All the pair plots have been generated in 4x4 matrix, each color-separated by the class labels. Most of the scatter plots have less overlap between both classes, while some have almost none. Hence we can conclude that the data is linearly separable !

8.4 Box-plots

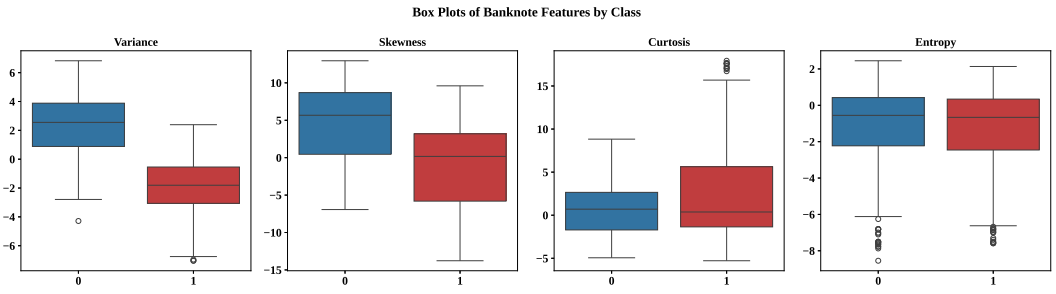

Figure 4: Box-plots of each feature grouped by class.

Interpretation: There is no overlapping between genuine and forged notes when it comes to variances while skewness has somewhat of an overlap. However the notes cannot be separated by looking at curtosis and entropy as both classes have almost full overlap in both features.

8.5 Training Error vs Epoch

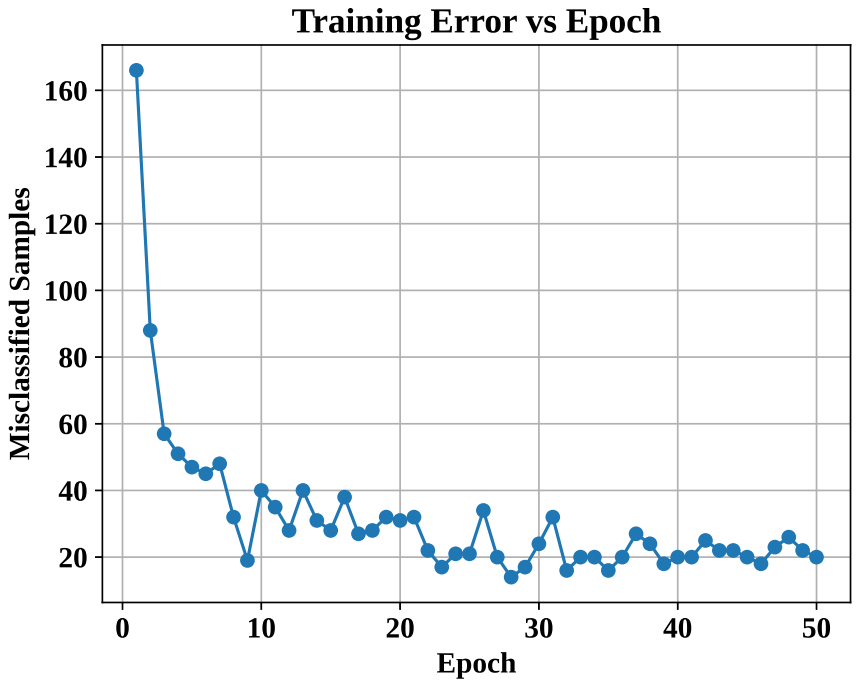

Figure 5: Number of misclassified training samples per epoch.

Interpretation: We observe the error (misclassified samples) reduce drastically after the first few epochs. The error of the perceptron later "stabilized" when about 30 epochs had been reached.

8.6 Weight Evolution

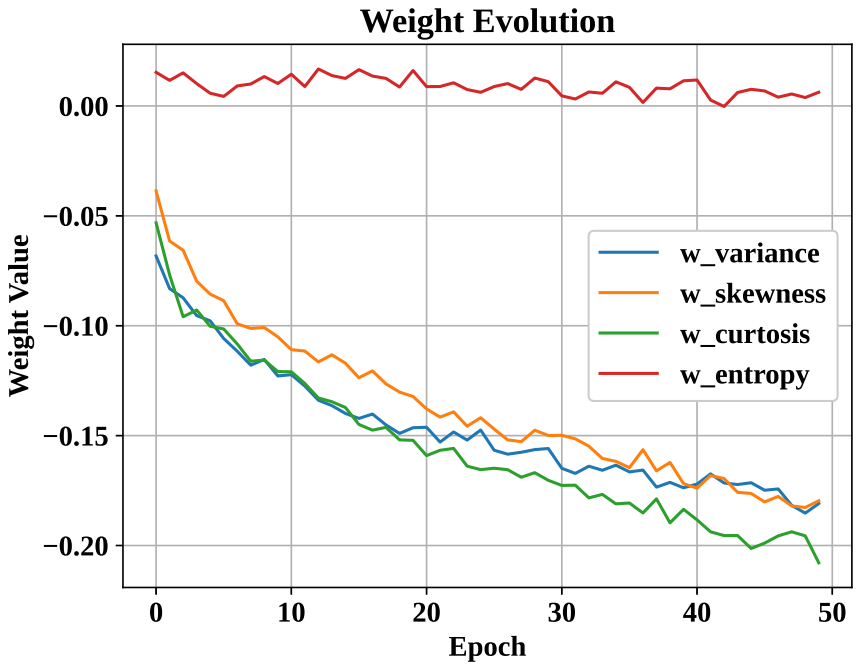

Figure 6: Evolution of the four feature weights across training epochs.

Interpretation: Weight values changed abruptly during the initial epochs. However the weight values later flat line when our model was able to find a good enough decision boundary between the target classes as the epochs grew.

8.7 Bias Evolution

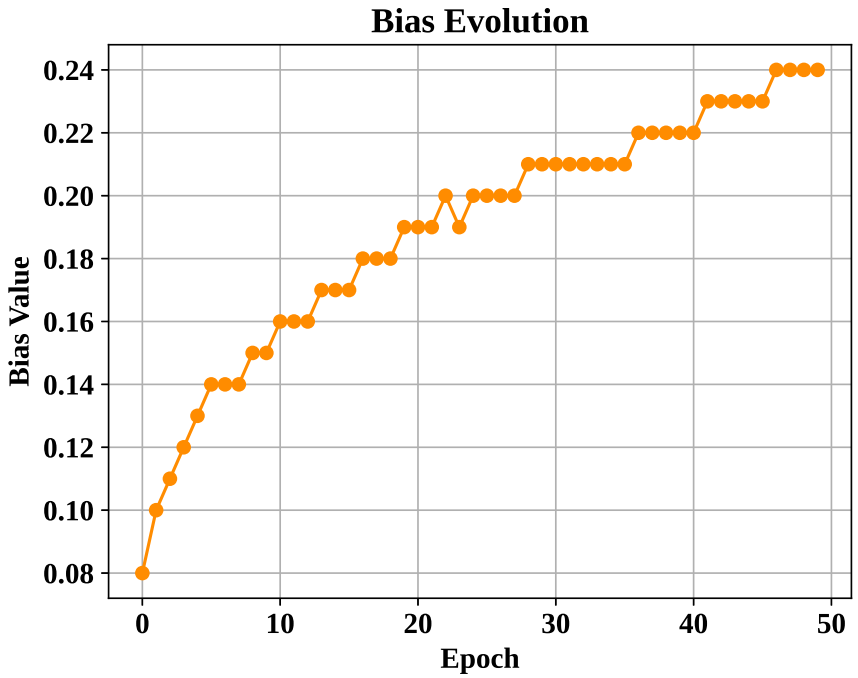

Figure 7: Evolution of the bias term across training epochs.

Interpretation: The bias value kept increasing significantly in the initial epochs. In the later epochs the increase wasn't as huge as the initial increase but the values still were constantly rising.

8.8 Confusion Matrix

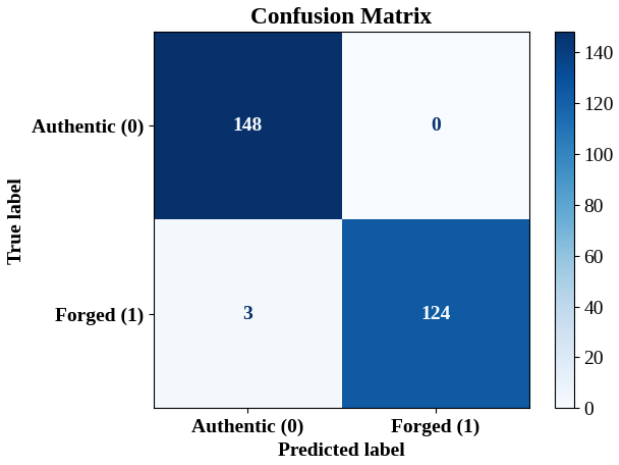

Figure 8: Confusion matrix on the test set.

Interpretation: Only 3 samples were wrongly classified as authentic bills, hence the slight reduction in accuracy. However, all the authentic bills were predicted correctly,hence the model has

100% precision.

8.9 Learning Rate Comparison

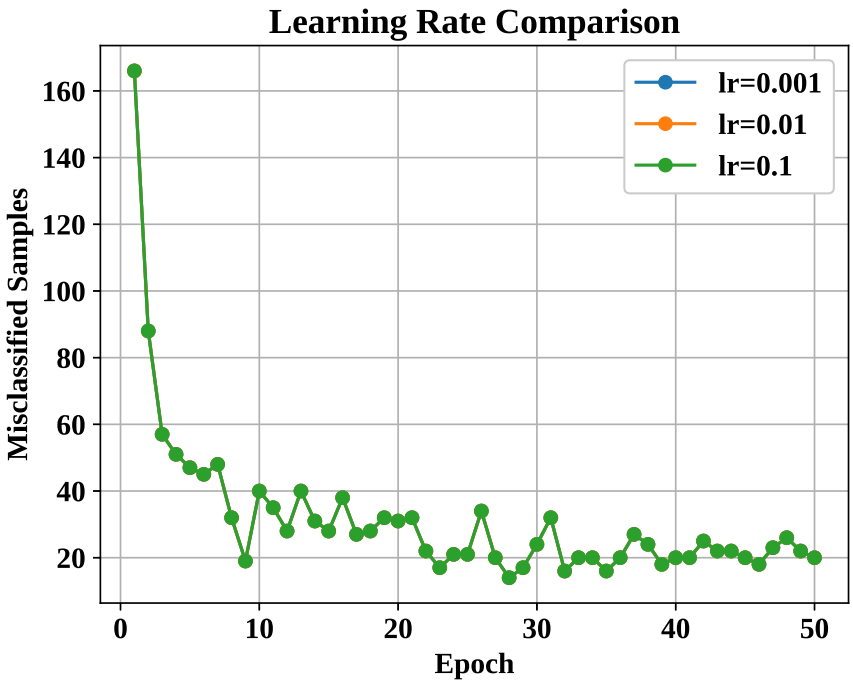

Figure 9: Training error curves for $\eta = 0.001, 0.01, 0.1$.

Interpretation: All three curves end up on top of each other. This is because at every epoch the same samples were misclassified as they had the same sign, even though they had different learning rates.

8.10 Decision Boundary (Optional)

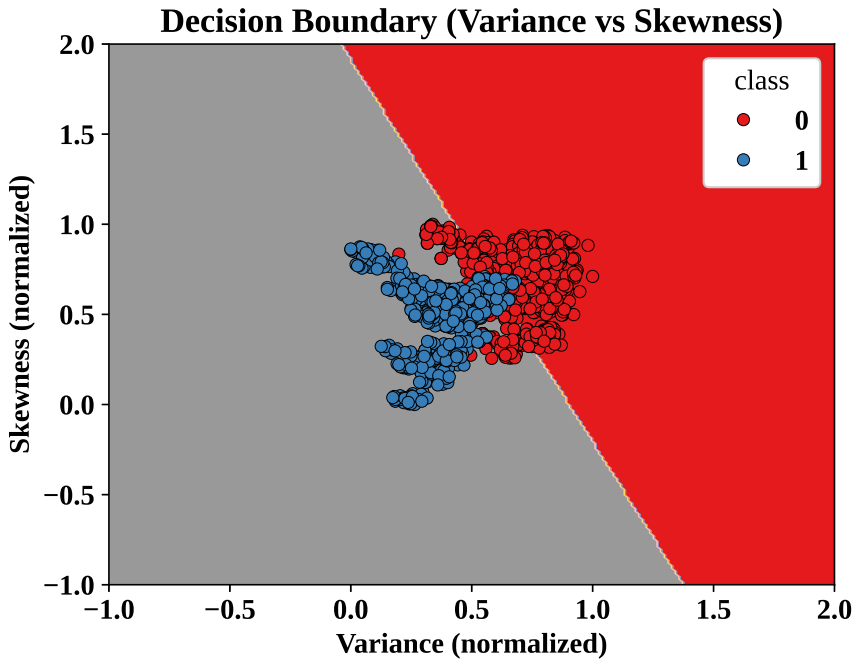

Figure 10: Decision boundary using two selected features (variance, skewness).

Interpretation: Variance and Skewness features were chosen and a linear decision boundary was drawn on their scatter plot. But since they are not linearly separable, it wasn't possible to separate both features using a linear boundary.

9. Performance Tables

Training Summary

Dataset Size	1372 samples
Train/Test Split	1097 / 275 (80% / 20%)
Learning Rate	0.01
Epochs	50 (did not fully converge to zero error)
Final Weights	$[-0.181,\ -0.180,\ -0.208,\ 0.006]$
Final Bias	0.240
Accuracy	0.9891
Precision	1.0000
Recall	0.9764
F1-score	0.9880

Epoch-wise Learning (selected epochs)

Epoch	Errors	Weight 1 (variance)	Weight 2 (skewness)	Bias
1	166	−0.068	−0.039	0.080
10	40	−0.123	−0.105	0.150
20	31	−0.146	−0.132	0.190
30	24	−0.156	−0.150	0.210
40	20	−0.174	−0.172	0.220
50	20	−0.181	−0.180	0.240

10. Discussion

Effect of Learning Rate. Repeating training with $\eta = 0.001, 0.01, 0.1$ produced identical error curves and identical final test accuracy for all three rates. This is a direct mathematical consequence of the chosen implementation: since the weights and bias are initialized to zero and training samples are processed in the same fixed order every epoch (no shuffling), scaling η by a constant only scales the magnitude of the weight vector – it does not change the *sign* of $\mathbf{w}^T\mathbf{x} + b$ at any step, and therefore does not change which samples are misclassified. This invariance would not hold if the data order were shuffled between epochs or if the weights were randomly initialized.

Step vs. Sigmoid Activation. Step function outputs a constant number and hence gives zero gradient everywhere, making it unsuitable for back-propogation. Whereas, the sigmoid is smooth and differentiable everywhere. Due to this Step function is resereved just for classic single layer perceptron, while deep networks use functions like Sigmoid, ReLu and Tanh.

Why a Single Layer Perceptron Cannot Solve XOR. XOR is a non linear plot. Hence it cannot be separated by a linear decision boundary (straight line). Hence we require atleast one hidden layer to add non-linearity to our perceptron and hence solve the XOR problem.

Effect of Feature Normalization. Without normalization, features with larger numeric ranges dominate the perceptron's weight updates, since each update is proportional to the raw input value $\mathbf{x}$. Normalizing all features to a comparable scale prevents any single feature from dominating training purely due to its scale, resulting in a more stable and balanced convergence.

From-scratch vs. Scikit-learn Perceptron. The from-scratch implementation achieved a marginally higher test accuracy (0.9891) than scikit-learn's built-in `Perceptron` (0.9782) on this split. Scikit-learn's `Perceptron` shuffles training data every epoch and applies an early-stopping criterion, while the from-scratch version iterates over a fixed sample order for a fixed number of epochs; these implementation-level differences lead to slightly different final decision boundaries and, consequently, small accuracy differences on a limited test set.

11. Conclusion

A Single Layer Perceptron was implemented entirely from scratch – including weight/bias initialization, the step activation function, forward propagation, and the perceptron learning rule – and trained on the Banknote Authentication dataset. After normalization and an 80:20 train–test split, the model's training error dropped sharply within the first few epochs and then plateaued at a low but non-zero level, reflecting that the dataset is close to, but not perfectly, linearly separable. The

final model achieved a test accuracy of 98.91% with perfect precision, confirming that a single linear decision boundary is sufficient to separate the two classes with high reliability. The learning-rate comparison showed that, under this fixed-order, zero-initialized implementation, the choice of η affects only the magnitude of the learned parameters and not the classification outcome. Comparison with scikit-learn's `Perceptron` validated the correctness of the from-scratch implementation, with both models achieving accuracy above 97%.

12. References

1. F. Rosenblatt, "The Perceptron," *Psychological Review*, 1958.
2. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.
3. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
4. S. Haykin, *Neural Networks and Learning Machines*, Pearson, 2009.
5. UCI Machine Learning Repository – Banknote Authentication Dataset.
6. Scikit-learn Documentation: Perceptron.